

GRAD Project

Presentation of the GRAD Project and its components

Components

- GRAD Machine
A CPU implemented in the "Digital" circuit simulator
- GRAD Language
A programming language designed for the GMachine
- GRAD Compiler
Software that compiles GLang source code into GMachine instructions

GRAD Machine

Architecture, instructions, and registers

Overview

- 16-bit von Neumann architecture processor
- 16-bit word and byte size, no endianness
- Possesses I/O for interacting with the user
- Two types of instructions
- Four registers across three types

Instruction types

- Compute instructions (C-instructions)
Used for computing values, performing comparisons, and jumping to instructions
- Address instructions (A-instructions)
Used for loading constants

Registers

- D (Data) register
General purpose, mainly used for calculations
- A (Address) register
Stores values loaded from A-instructions, also used for indexing into memory
- ONE and TWO registers
Supply 1 and 2 as constants, reducing the number of A-instructions needed

GRAD Language

Statements and complex features

Overview

- Designed specifically for the GRAD Machine
- Hybrid language, same syntax for both assembly and low level statements
- Each non-blank line corresponds to one statement
- Loops and conditionals performed through jumps
- Functions are not supported
- Four types of statements

Types of statements

- Alias declarations
Binds an identifier to a numeric constant, or another alias
Treated like regular numeric constant usage
- Variable declarations
Binds an identifier to a location in memory
Treated like an access to memory

Types of statements (cont.)

- Instructions

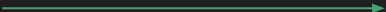
Runs an A or C instruction in the GMachine

- Jump label declarations

Provides a line-number agnostic method to jump to a specific point in the program

Complex features

- Used to increase code readability by allowing inline use of constants
- Instructions using complex features must be "decompressed" into multiple assembly/simple instructions
- Decompression heavily uses the D register

$M[256] := D + M[128]$  $@128$
 $D := D + M$
 $@256$
 $M := D$

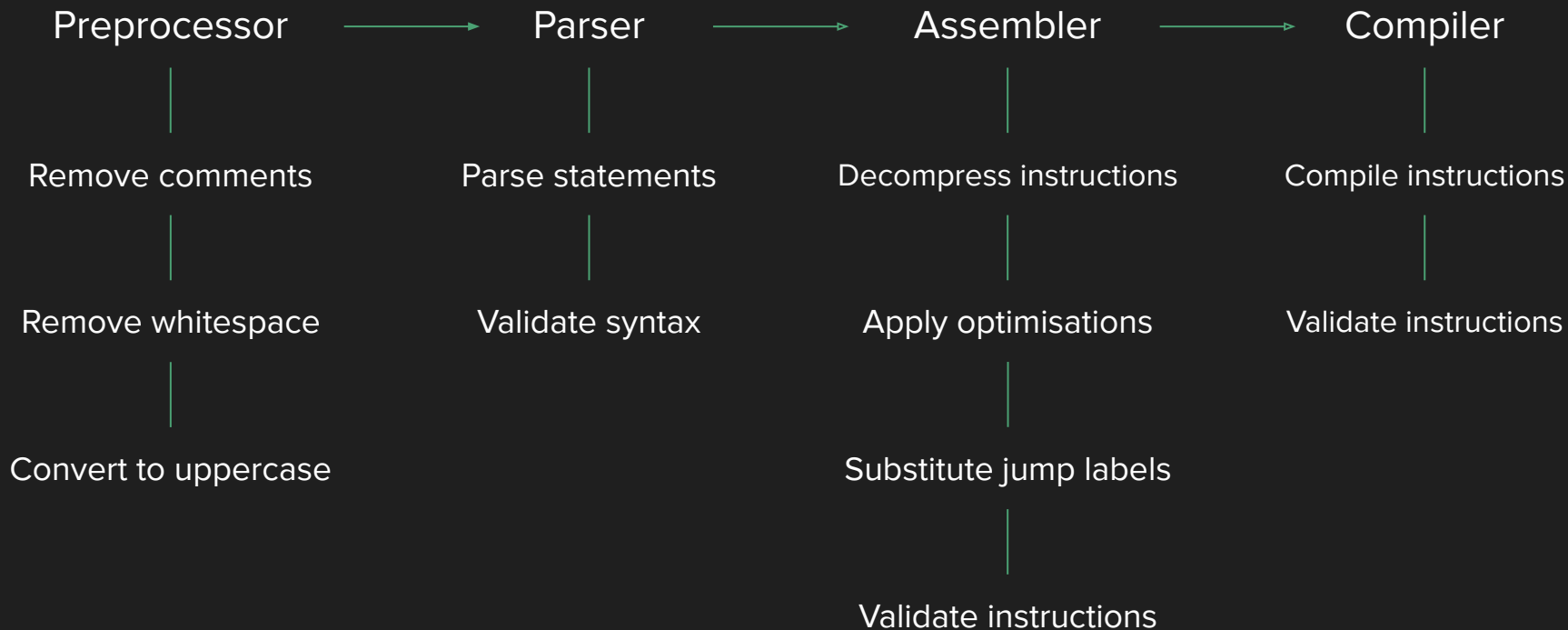
GRAD Compiler

Compilation, decompression, and optimisation

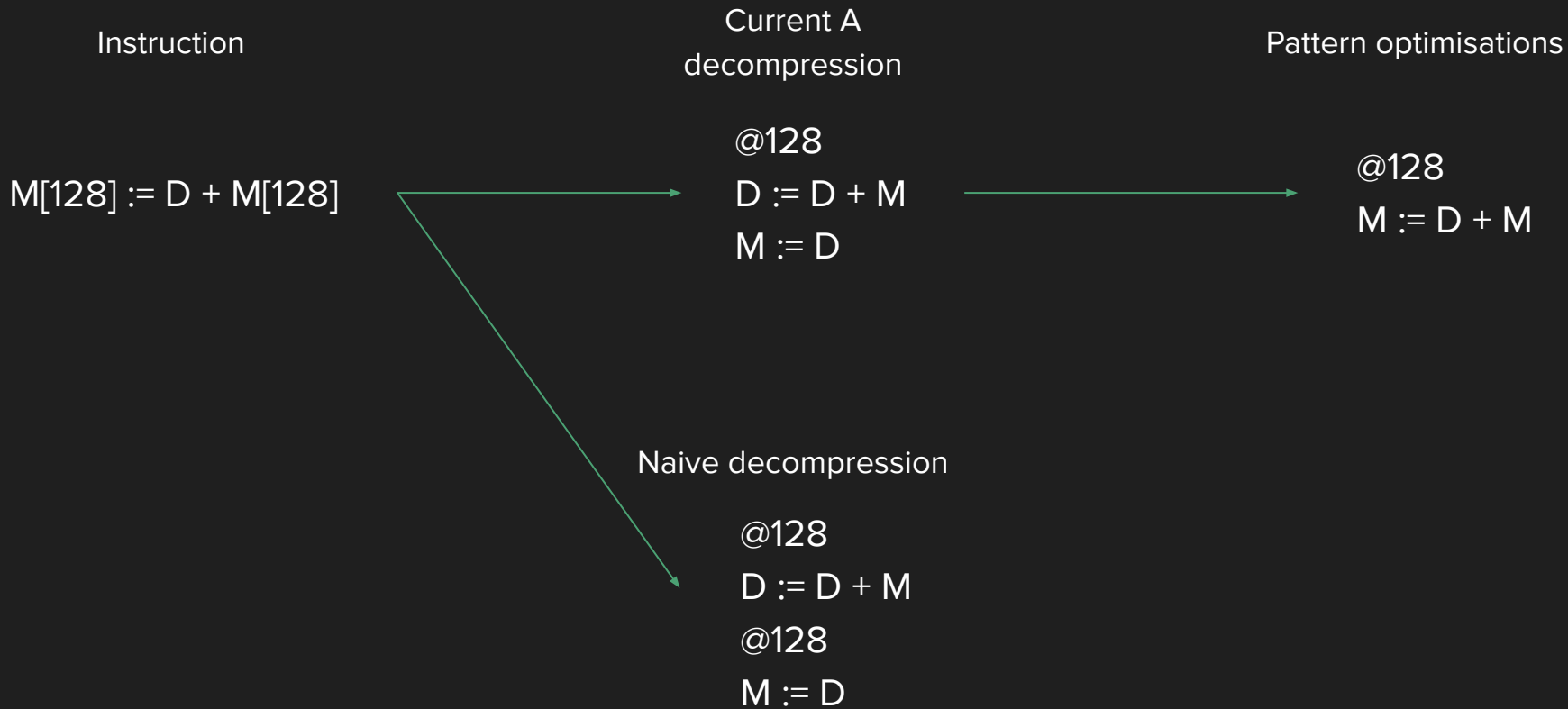
Overview

- Written in Python
- Multi-pass assembler and compiler
- Four stage compilation pipeline
- Configurable behaviour to assist in debugging
- Continues processing after detecting errors for more complete reports

Compiler pipeline



Optimisations



Thank you for your time